Low-Level Design (LLD)

MedInfo Assistant - Medical Research & Report Analysis Multi-Agent GenAI Platform

Generated 2026-10-15 from 04-LLD.md - information only, not medical advice.

Contents

4.0 Document control

4.1 Chat pipeline (rag/service.py: ChatService)

4.2 Retrieval (rag/retrieval.py, rag/rerank.py, rag/versions.py, rag/providers.py)

4.3 Ingestion (rag/ingest.py, loaders.py, ocr.py, chunking.py)

4.4 Resilience (rag/fallback.py, core/circuit_breaker.py, core/rate_limit.py)

4.5 Memory, cache, review and report

4.6 Error model

4.7 Authentication and authorisation (core/auth.py, api/deps.py)

4.8 Guardrail engine (guardrails/engine.py, policy.yaml)

4.9 Triage and scope rules

4.10 Answer checks (safety/answer_checks.py)

4.11 Prompts (rag/prompts.py)

4.12 API layer and SSE protocol

4.13 Frontend modules (frontend/src/app.js)

4.14 Configuration reference

4.15 Metrics reference (core/metrics.py)

4.16 Test seams (backend/tests/fakes.py)

4.0 Document control

Field	Value
Document	Low-Level Design
Version	2.0 (production-grade)
Status	Draft for approval
Date	2026-10-14
Inputs	PRD, HLD, Architecture
Code root	`backend/app/` , `frontend/src/`

Section numbers (4.x) are referenced from the PRD traceability matrix.

4.1 Chat pipeline (`rag/service.py: ChatService`)

4.1.1 Public API

```
ChatService.answer(question: str, *, user_id: str, filters: SearchFilter | None = None,
                   history: list[tuple[str, str]] | None = None, conversation_id: str | None =
None,
                   research: bool = False, capture: list[Passage] | None = None,
                   on_phase: Callable[[str], None] | None = None) -> ChatAnswer
```

4.1.2 Steps

#	Step	Implementation	On failure
1	Validate	Strip; empty or > `MAX_QUESTION_CHARS` -> `InputRejectedError` (422)	422
2	Phase `safety`	`on_phase("safety")`	-
3	Input guardrails	`PolicyEngine.check_input(q)` -> `GuardResult(text=q_redacted, action, events)`	Block -> refusal (step 6)
4	Conversation	`MemoryService.start_or_get(conversation_id, user_id, q_redacted)` ; history only when continuing	404 if not owner
5	Triage	`TriageService.assess(raw, llm_text=q_redacted)`	LLM failure -> rules result stands
6	Crisis / input block	Crisis -> emergency answer only. Input block -> refusal (+ emergency notice if triage fired)	-
7	Scope	`ScopeClassifier.classify(q_redacted)` -> most restrictive of rules/LLM	LLM failure -> rules
8	Fixed paths	UNSAFE -> refusal with support; OUT_OF_SCOPE -> redirect; EMERGENCY (scope) adds the notice	-
9	Cache lookup	Only first turn, not research, no emergency: `AnswerCache.key(q, scope, filters, user)`	Cache errors -> miss
10	Phase `searching` ; graph	`graph.stream(inputs, stream_mode="updates")` ; on `retrieve` node -> phase `generating`	Provider error: if emergency -> return notice; else raise (429/502/503)
11	Not covered	No passages or `NOT_COVERED` -> not-covered answer (dosing: + prescribing refusal)	-
12	Phase `verifying` ; split	`split_limitations(raw)` (inline or own-line `LIMITATIONS:`)	-

#	Step	Implementation	On failure
13	Output guardrails	`check_output(body)` : prompt leak -> block; PHI/secrets -> redact	Block -> refusal
14	Verification	`AnswerChecker.run(body, passages, high_risk)` (4.10)	Any failure -> refusal template
15	Assemble	Preface (referral / cite-only intro), answer, warnings (medicine, version conflict, staleness), citations, evidence basis, limitations, disclaimer	-
16	Post-process	Cache if `AnswerCache.cacheable` ; `ReviewService.maybe_sample` ; memory record with citation provenance; audit + latency metric	Memory/review failures logged, never surfaced

4.1.3 Scope -> answer mode

Scope	Prompt mode	Extra wording
info, research, emergency	STANDARD	-
personal_diagnosis	EDUCATIONAL	Diagnosis referral preface
prescribing_or_dosing	QUOTE_ONLY	Cite-only intro + medicine-change warning
unsafe / out_of_scope	(no generation)	Refusal / redirect

4.1.4 Agent trace (`rag/trace.py: AgentTrace`)

"How this answer was produced". Status: **Implemented + tested offline** (`tests/test_agent_trace.py` , 8 tests incl. a no-PHI test; 2 browser tests in `frontend/tests/ui.spec.js`).

- `AgentTrace.step(agent, outcome, detail="", status="ok")` appends a `TraceStep{agent, outcome, detail, status, ms}` ; `ms` is the time since the previous step (`restart_clock()` before the LangGraph stream).
- Steps, in order, as they run: `Input guardrails` , `Triage agent` , `Scope agent` , `Answer cache` (only when the cache is eligible), `Query rewriter` , `Retriever + re-ranker` or `Research agent (ReAct)` , `Answer agent` (graph nodes via `ChatService._trace_node`), `Output guardrails` , `Verifier` , and a final `Result` (answer kind + scope). Early exits (crisis, input block, unsafe/out-of-scope, cache hit, not covered, output block, verifier block) end the trace at the stopping step followed by `Result` .
- Status: `ok` , `warn` (e.g. redaction, diagnosis sentences removed, nothing retrieved), `stop` (emergency/crisis, block, unsafe/out-of-scope, verifier refusal), `skip` (e.g. rewriter on a first question, cache miss).
- Content rule: decisions, rule names, class names and counts only - never question text, answer text, passage text or upload filenames (the trace is returned to the browser). On a cache hit the trace of the current request replaces the cached one. The trace is not stored in conversation memory.
- Returned as `ChatAnswer.safety.trace` (API spec); rendered by `renderTrace()` (4.13).

4.2 Retrieval (`rag/retrieval.py`, `rag/rerank.py`, `rag/versions.py`, `rag/providers.py`)

4.2.1 Hybrid search per query

1. `embed_query(q)` (EURI, 1536 dims).
2. For each collection in {trusted} ∪ ({user} if `owner_id` and not `trusted_only`): - **Vector:** `1 - (embedding <=> :vector)`, filtered, top `RETRIEVAL_CANDIDATES` (12), keep score >= `RETRIEVAL_MIN_SIMILARITY` (0.25). - **Keyword (BM25-style):** query lexemes from `to_tsvector('english', :query)`; per-term document frequency over the filtered collection; score = Σ ln(1 + (N - df + 0.5)/(df + 0.5)) · tf·2.2 / (tf + 1.2·(0.25 + 0.75·dl/avgdl)) using the generated `tsv` column (GIN index).
3. **RRF:** score(d) = Σ_rankings 1/(60 + rank).
4. Top 12 fused -> `chunk_to_passage` (parent expansion for parent-child) -> de-duplicate siblings.
5. **Re-rank:** LLM returns `{"relevant":[n,...]}` (at most k); invalid/duplicate numbers ignored; error -> fused order.
6. **Version preference:** series key = `series` metadata or title without version/year tokens; older versions marked `superseded`, placed after newer; note "may conflict; newest preferred".

4.2.2 Filters (SQL, parameterised)

`trusted IS TRUE OR owner_id = :owner` (owner scoping always enforced) · `publisher IN (...)` · `doc_id IN (...)` · `doc_type IN (...)` · `jurisdiction = :j` · `publication_date >= :from / <= :to`.

4.2.3 Multi-query

Only with history: `{"standalone": ..., "alternatives": [...]}` (max 2) -> all queries searched, fused.

4.3 Ingestion (`rag/ingest.py` , `loaders.py` , `ocr.py` , `chunking.py`)

4.3.1 Loaders

Type	Unit	Location kept	Notes
PDF	Page	`page`	< 25 chars of text -> OCR of embedded images
DOCX	Heading block / table	`section`	Tables -> table sections
PPTX	Slide (+ tables)	`slide` , `section` (title)	
XLSX	Sheet	`sheet`	Rows -> table section
CSV/TSV	File	-	UTF-8-SIG / cp1252 fallback
TXT/MD	File	-	CRLF -> LF
PNG/JPG	Image	`page=1` , `ocr`	OCR required

OCR: RapidOCR (ONNX, offline), grayscale + autocontrast + 40 px border, `det_unclip_ratio=2.0` (tuned: default dropped lines and merged words), min confidence 0.5, line grouping by vertical position.

4.3.2 Pipeline

1. Size (`MAX_UPLOAD_MB`), type, filename sanitised (`Path(name).name`).
2. Trusted: publisher must be in `trusted_sources.yaml` ; licence/jurisdiction defaulted from the list.
3. `doc_id = "doc_" + sha256(collection, owner, filename)[:24]` .
4. `file_hash = sha256(normalised bytes + provenance + chunker config)` ; equal -> `unchanged` .
5. Load -> **document guardrails** (neutralise injection lines, redact PII/secrets, flag section).
6. Chunk: recursive (1000 chars, 15%), semantic (sentence-embedding breakpoints at the 90th percentile distance, 15%), parent-child (2000/400, children 10%), tables always table-aware (header repeated, rows never split).
7. `chunk_hash = sha256(text, kind, location, provenance, chunker config)` ; ids = UUID(sha256(doc_id, hash, occurrence)) - position independent.
8. Same hashes but different bytes -> `unchanged` (registry updated, parents preserved).
9. Embed new/changed chunks in batches of 64 -> upsert -> delete stale ids -> registry put -> bump corpus/user version (cache invalidation) -> audit.

4.4 Resilience (`rag/fallback.py`, `core/circuit_breaker.py`, `core/rate_limit.py`)

Mechanism	Parameters	Behaviour
Fallback chain	`LLM_ANSWER_CHAIN`	Try in order; skip open circuits
Retry	`MODEL_RETRY_ATTEMPTS` =2, base 0.5 s, ×2	Only `ProviderUnavailableError`
Circuit breaker	threshold 3, reset 30 s	CLOSED -> OPEN after 3 failures; HALF_OPEN after 30 s (1 trial)
Result	-	None attempted -> 503 "circuits open"; all quota -> 429; else last non-quota error
Rate limits	chat 20/min, research 20/h, upload 30/h	Fixed window in Redis; 429 + Retry-After; limiter outage -> allow (logged)
Reasoning models	gpt-oss, gemini-3*, o3/o4	`reasoning_effort="low"`

4.4.1 LLM gateway: routing, token limits, accounting (`rag/gateway.py`, `rag/llm_gateway.yaml`) - ADR-006

Every chat-model call goes through `LLMGateway.for_task(task)`, which returns a `TaskModel` (same `ChatModel` interface). Status: **implemented and tested offline**; real token numbers not yet measured live.

Task	Route	Min / max output tokens	Max input tokens (estimated)	On failure or over budget
triage	fast	32 / 80	1500	rules decide (fail open to rules)
scope	fast	16 / 40	1500	rules decide
rewrite	fast	64 / 160	2500	original question used
rerank	fast	32 / 80	4000	fused RRF order used
judge	quality	64 / 120	6000	**refuse (fail closed)**
answer	quality	256 / 1200	6000	lowest-ranked passages dropped first; none fit -> error (emergency notice kept)
eval_judge	quality	64 / 300	8000	metric scores 0

- Routes: `quality` = `LLM_ANSWER_CHAIN`; `fast` = `LLM_FAST_CHAIN` if set, else the answer chain. Each route is a `FallbackChatModel` with circuit breakers.
- Output budget: `clamp(requested or max, min, max)`. The minimum exists because reasoning models return empty text when `max_tokens` is tiny.
- Input budget: `ceil(len(text) / chars_per_token)` (4), summed over system + user prompt. Over budget -> `TokenBudgetExceededError` before any provider call.

- `fit_passages` (in `agents/rag_graph.py`) pops passages from the end (lowest rank) until the answer prompt fits, and the `generate` node returns the kept list as `passages`.
- Accounting: `current_task` context variable; the provider adapter records `usage_metadata` as `medinfo_llm_tokens_total{task,model,direction}` (absent on cached replies).
- Query-embedding cache (`rag/cache.py: CachedEmbedder`): key `medinfo:emb:sha256(model, dims, text)`, value = the vector; `embed_documents` is never cached (ingestion already skips unchanged chunks by content hash); KV failure falls through to the model.

4.5 Memory, cache, review and report

Component	Key design
`ShortTermMemory`	Redis list `medinfo:conv:<id>:turns`, max `SHORT_TERM_TURNS`, TTL 2 h; miss -> long-term store
`SqlConversationStore`	Titles and text Fernet-encrypted (`CONVERSATION_ENCRYPTION_KEY`); owner filter in every query
Retention	`purge_older_than(now - CONVERSATION_RETENTION_DAYS)`
`AnswerCache`	Key includes corpus version and per-user upload version; stores only `kind=answer`, no emergency, no guardrail events, no history, all checks passed
`SafeLLMCache`	Stores `[{"type":"ai","text":...}]`; anything else is a miss; tool-call replies not cached
`ReviewService`	Samples `kind=answer` at `REVIEW_SAMPLE_RATE`; verdicts safe_and_correct / minor_issue / incorrect / unsafe
Report	`build_markdown(conv, messages, emergency_number)`; HTML-escaped text; citation provenance from message meta

4.6 Error model

Error	HTTP	Raised when
`InputRejectedError`	422	Empty/too long question
`UnsupportedFileTypeError`	415	Unknown extension
`DocumentTooLargeError`	413	Upload over limit
`UntrustedPublisherError`	422	Publisher not allow-listed
`DocumentNotFoundError`, `ConversationNotFoundError`, `ReviewItemNotFoundError`	404	Missing **or not the caller's**
`RateLimitedError`	429	Per-user limit (+ Retry-After)
`ProviderQuotaError`	429	Provider quota/rate limit, incl. 403/402 with quota wording
`ProviderError`	502	Provider error
`ProviderUnavailableError`	503	Unreachable, timeout, not configured, all circuits open
Unhandled	500	Logged with traceback (explicit `exc_info`)

4.7 Authentication and authorisation (`core/auth.py` , `api/deps.py`)

1. `Authorization: Bearer <jwt>` ; missing -> 401.
2. Header `alg` must be ES256 or RS256 (HS256/none rejected before key lookup).
3. Key from JWKS (`PyJWKClient` , cached `JWKS_CACHE_SECONDS`).
4. `jwt.decode(..., audience="authenticated", issuer=<supabase>/auth/v1, require exp/sub/aud, leeway=JWT_CLOCK_SKEW_SECONDS (30))` . The leeway fixes a live-observed `ImmatureSignatureError` caused by local clock skew.
5. Roles: `app_metadata.roles[]` / `app_metadata.role` ∩ {user, clinician, admin}; default `user` ; top-level `role` ignored. Rank: user < clinician < admin.
6. `require_user/clinician/admin` dependencies; user id stored on `request.state` for the audit line.

4.8 Guardrail engine (`guardrails/engine.py`, `policy.yaml`)

- Stages: `input`, `document`, `output`; actions ALLOW < FLAG < REDACT < BLOCK (max wins).
- Rule groups: `prompt_injection`, `jailbreak`, `unsafe_topic`, `secrets`, `prompt_leak`; PII entities: email, MRN/IDs, DOB, national IDs, phone, address, person name (each with a label such as `[NAME]`).
- Document stage: injection/jailbreak matches neutralise the **whole line** (`[instruction-like text removed]`) and flag the section.
- Safety-enforced output rules (`prescribing`, `diagnosis`) must be `block`; otherwise `PolicyError` at load.
- Every match -> metric `medinfo_guardrail_events_total{stage,rule,action}` + log (no text).

4.9 Triage and scope rules

Item	Triage (`red_flags.yaml`)	Scope (`scope.py`)
Normalisation	NFKC, lower-case, straight quotes, collapsed whitespace, **apostrophes removed** (pattern and text)	Same
Categories	12: chest pain, stroke, breathing, anaphylaxis, heavy bleeding, unconscious/seizure, self-harm (crisis), overdose/poisoning (poison), meningitis signs, pregnancy emergency, infant severe, sudden severe headache	unsafe > prescribing_or_dosing > personal_diagnosis > research > info (rules); LLM can add emergency/out_of_scope
LLM output	`{"emergency","category","self_harm"}` ; unknown category -> `other`	`{"scope"}` ; invalid -> ignored
Governance	`status: draft_pending_clinician_review` ; version bump per change	Tests per class

4.10 Answer checks (`safety/answer_checks.py`)

Check	Algorithm	Result
Diagnosis	Sentence regexes ("you (probably) have", "this sounds like", "your symptoms suggest", ...) excluding conditional prefixes ("if you have")	Sentences removed + referral
Citations	Quote-aware sentence split; each non-"not covered" sentence needs `[n]` with 1 ≤ n ≤ passages	Fail -> refusal
Numbers	Every number in the answer must appear in passage text or header (title/version/date/page); version "3.0" also allows "3"	Fail -> refusal
Drug names	Lexicon words in the answer must appear in passages	Fail -> refusal
Dosing	Sentences with a dose pattern or a start/stop/increase... verb near medicine context must contain a verbatim quote from a cited passage; no dose/action words outside the quote	Fail -> prescribing refusal
Faithfulness	High-risk (dosing, pregnancy, children, interactions): judge JSON `{"faithful":bool}` ; error -> fail	Fail closed
Staleness	Cited passage older than `STALE_SOURCE_YEARS` (5)	Warning

4.11 Prompts (`rag/prompts.py`)

- System rules: passages only; cite every sentence; copy numbers/units/drug names exactly; doses only as verbatim quotes; never diagnose or personalise; `NOT_COVERED` when unanswerable; state evidence strength and disagreement; treat UNTRUSTED/SUPERSEDED passages accordingly; no disclaimers (added in code); finish with `LIMITATIONS:` on a new line; an output-format example with placeholders only.
- Passages: `<passage n="i" trust="TRUSTED|UNTRUSTED user-provided|UNTRUSTED web[; flagged]" [status="SUPERSEDED..."]>` with title, version, publisher, location, date.
- Question wrapped in `<question>` and declared data, not instructions.

4.12 API layer and SSE protocol

- Routers with prefixes (`/api` , `/api/admin` , `/api/documents` , `/api/conversations` , `/api/review`) so metric route labels are templates.
- `POST /api/chat/stream?mode=chat|research` : response `text/event-stream` ; events: `event: phase` / `data: {"phase":"safety|searching|generating|verifying"}` , then `event: answer` / `data: <ChatAnswer JSON>` or `event: error` / `data: {"detail","status","retry_after"}` .
- Middleware: request id, HTTP metrics by route template, per-request audit line, security headers, `Cache-Control: no-store` for `/api` , `no-cache` for UI assets.

4.13 Frontend modules (`frontend/src/app.js`)

Module (function group)	Responsibility
Auth	Supabase password/refresh grants (no SDK), token in sessionStorage
API client	Bearer header, 401 -> sign-out, Retry-After messages
Streaming	`fetch` + manual SSE parser; phase list UI
Rendering	`renderAnswer` (emergency card, refusal card, cited paragraphs with chip buttons, source cards, limitations, confidence note, agent-trace panel via `renderTrace`, disclaimer) - text nodes only
History / report	Conversation list, open, delete all, report download (blob)
Admin / clinician	Documents upload (drag-and-drop) and table, evaluation rings (SVG), review queue
Shell	Theme toggle, mobile drawer, role-gated tabs, boot message

4.14 Configuration reference

All settings are environment variables (`.env` at the repository root; documented in `.env.example`).

Setting	Default
`APP_ENV`	local (production enforces required settings)
`LOG_LEVEL` / `LOG_FORMAT`	INFO / text
`AUDIT_LOG_ENABLED`	true
`SUPABASE_URL` , `SUPABASE_PUBLISHABLE_KEY`	- (publishable key is public by design)
`SUPABASE_SECRET_KEY`	secret; admin scripts only
`SUPABASE_JWT_AUDIENCE`	authenticated
`JWKS_CACHE_SECONDS` / `JWT_CLOCK_SKEW_SECONDS`	600 / 30
`DATABASE_URL` , `REDIS_URL`	secrets
`EURI_*` , `GEMINI_*` , `GROQ_*` (`_API_KEY` , `_BASE_URL`)	keys secret; base URLs preset
`LLM_ANSWER_CHAIN` / `LLM_AGENT_CHAIN`	euri:gpt-4.1-mini
`LLM_FAST_CHAIN`	empty (= answer chain); gateway route for triage, scope, rewrite, re-rank
`LLM_TIMEOUT_SECONDS` / `LLM_MAX_TOKENS` / `LLM_TEMPERATURE`	45 / 1200 / 0.2 (0.1-0.3 enforced)
`EMBEDDING_PROVIDER` / `EMBEDDING_MODEL` / `EMBEDDING_DIMENSIONS`	euri / text-embedding-3-small / 1536
`TAVILY_API_KEY` , `RESEARCH_ENABLED` , `WEB_ALLOWED_DOMAINS`	secret / true / who.int, nice.org.uk, cdc.gov, ...
`EMERGENCY_NUMBER` (+ service, crisis, poison)	112 (must be explicit in production)
`STALE_SOURCE_YEARS` / `MAX_QUESTION_CHARS`	5 / 2000
`RETRIEVAL_BACKEND`	auto (pgvector when `DATABASE_URL` is set)
`RETRIEVAL_TOP_K` / `RETRIEVAL_CANDIDATES` / `RETRIEVAL_MIN_SIMILARITY` / `RERANK_ENABLED`	5 / 12 / 0.25 / true
`COLLECTION_PREFIX`	medinfo
`CHUNK_STRATEGY` / `CHUNK_SIZE` / `PARENT_CHUNK_SIZE` / `CHILD_CHUNK_SIZE` / `MAX_UPLOAD_MB`	recursive / 1000 / 2000 / 400 / 10
`ANSWER_CACHE_TTL_SECONDS` / `LLM_CACHE_TTL_SECONDS` / `EMBEDDING_CACHE_TTL_SECONDS`	3600 / 86400 / 604800

Setting	Default
`CONVERSATION_RETENTION_DAYS` / `CONVERSATION_ENCRYPTION_KEY` / `SHORT_TERM_TURNS`	30 / secret (required in production) / 10
`RATE_LIMIT_CHAT_PER_MINUTE` / `_RESEARCH_PER_HOUR` / `_UPLOAD_PER_HOUR`	20 / 20 / 30
`CIRCUIT_FAILURE_THRESHOLD` / `CIRCUIT_RESET_SECONDS` / `MODEL_RETRY_ATTEMPTS` / `MODEL_RETRY_BASE_DELAY`	3 / 30 / 2 / 0.5
`REVIEW_SAMPLE_RATE` / `METRICS_TOKEN` / `FRONTEND_DIR`	0.05 / secret (optional) / auto-detected

4.15 Metrics reference (`core/metrics.py`)

Metric	Type	Labels	Meaning
`medinfo_http_requests_total`	Counter	method, route, status	HTTP requests (route = template)
`medinfo_http_request_seconds`	Histogram	method, route	Latency; p50/p95/p99 via histogram_quantile
`medinfo_chat_seconds`	Histogram	outcome	End-to-end chat latency by answer kind
`medinfo_model_call_seconds`	Histogram	model, outcome	Model call latency
`medinfo_retrieval_seconds`	Histogram	stage	embed_query / vector / keyword / rerank
`medinfo_triage_events_total`	Counter	category, detector	Triage firings
`medinfo_scope_decisions_total`	Counter	scope	Scope outcomes
`medinfo_refusals_total`	Counter	scope, reason	Refused/replaced answers
`medinfo_safety_check_failures_total`	Counter	check	Post-answer check failures
`medinfo_hallucination_flags_total`	Counter	check	Grounding/faithfulness failures
`medinfo_guardrail_events_total`	Counter	stage, rule, action	Guardrail matches
`medinfo_classifier_errors_total`	Counter	classifier	LLM classifier failures (fail-open)
`medinfo_cache_events_total`	Counter	cache, result	Cache hits/misses
`medinfo_circuit_state`	Gauge	model	0 closed, 1 half-open, 2 open
`medinfo_model_fallbacks_total`	Counter	model, error	Fall-through to next model
`medinfo_rate_limited_total`	Counter	bucket	Per-user limit rejections
`medinfo_llm_prompt_tokens_estimated`	Histogram	task	Estimated prompt tokens per call (budget check)
`medinfo_llm_tokens_total`	Counter	task, model, direction	Provider-reported tokens (input/output)
`medinfo_token_budget_events_total`	Counter	task, action	output_clamped / input_rejected / passages_trimmed

All labels are bounded enumerations; no free text or PHI.

4.16 Test seams (`backend/tests/fakes.py`)

Fake	Replaces	Notes
`FakeChatModel`	All chat calls	Routes by prompt type (triage, scope, judge, rerank, multi-query, answer); records every prompt for PHI checks
`FakeEmbedder`	Embeddings	Hashed bag-of-words; records embedded texts
`FakeOcr`	RapidOCR	Fixed text
`StaticKeyResolver`	JWKS	Locally generated EC/RSA keys
`FakeProbe`	DB probe	Ready/unready
`InMemoryVectorIndex` , `InMemoryRegistry` , `InProcessKV` , in-memory stores	Postgres, Redis	Same filter semantics as production
`ui_demo_server.py`	Whole backend for browser tests	Real app + fake model + synthetic data